Graphic Era deemed to be University
DEHRADUN

PROJECT AND TEAM INFORMATION

Project Title

Cross-Broker Real-Time Portfolio Aggregation System

Team Name: *Team #*	*Avengers*
Team member 1 (Team Lead) *(Last Name, name: student ID: email, picture):*	*Singh, Uday Raj – 2510010552* *udofficial789@gmail.com* 
Team member 2 *(Last Name, name: student ID: email, picture):*	*Jalal, Soumy – 2510370148* *soumyj12@gmail.com* 
Team member 3 *(Last Name, name: student ID: email, picture):*	*Jain, Saksham – 2510370127* *saksham82730@gmail.com* 
Team member 4 *(Last Name, name: student ID: email, picture):*	*Parth – 260211605* *p1rth2802@gmail.com* 

PHASE-II PROJECT PROGRESS DESCRIPTION

Project Abstract

Investors often hold stocks across several broker platforms such as Groww and Zerodha, so no single application shows the complete position. Our project, the Cross-Broker Real-Time Portfolio Aggregation System, collects the permitted holdings and trade information from multiple brokers, together with market prices and stock news, and presents everything through one unified dashboard. The dashboard shows stock names, quantities, invested amount, current value, profit/loss and trade history.

The system uses a hybrid design. A Python process handles network communication and data collection, and a C++ process handles structured processing using Object-Oriented Programming, pointers and dynamic memory. The two processes communicate through a local TCP socket.

Phase II covers the Python and C++ parts of the project: the data collection layer, the socket connection, the C++ class design, the parser and the portfolio calculations. The Linked List and Binary Search Tree work belongs to the Data Structures part of the project and is planned for Phase III.

Updated Project Approach and Architecture (2 pts)

Design. The system runs as two separate processes. The Python process collects broker, market and news data and formats it as text records. The C++ process receives these records, parses them and manages the portfolio.

Communication. The processes are connected by a local TCP socket on `127.0.0.1`. Each record is sent as one line of delimiter-separated text, for example `TRADE|broker|ticker|quantity|price|date`, so the C++ side can read it line by line.

Python side. The `socket` library is used for the connection and request for data collection. When live broker access is not available, representative sample data is used so that the full data flow can still be demonstrated.

C++ side. The incoming text is parsed with `std::stringstream`. Classes such as `Trade`, `Holding` and `Portfolio` represent the data and keep it encapsulated. Objects are created dynamically with `new`, accessed through pointers and released in destructors with `delete`.

Calculations. The `Portfolio` class computes the holding quantity, invested amount, current value and profit/loss for each stock and for each broker.

Tasks Completed

Task Completed	
Identified the broker, market and news fields needed and fixed the text record format	uday raj singh and parth
Python data collection and formatting module (with sample data fallback)	soumy jalal and parth
C++ classes for trades, holdings and portfolio, with dynamic memory and destructors	saksham jain and uday raj singh

Challenges/Roadblocks

1. **Broker data access.** Live broker APIs need authentication and may restrict access. We are using sample data in the same format for development and will add live data where permitted, without hard-coding credentials.

2. **Memory management.** Objects created with `new` must be freed correctly. We release them in destructors and check for leaks.

Tasks Pending

Task Pending	**Team Member (to complete the task)**
Singly Linked List for chronological trade records (Phase III)	uday raj singh
Binary Search Tree for ticker-based search (Phase III)	saksham jain
Integrate stock news and the one-year projection	soumy jalal
Broker-wise portfolio comparison and dashboard output	parth
Final testing with live or sample data and the final report	soumy jalal

Project Outcome/Deliverables

The final outcome is a unified dashboard that monitors investments across multiple broker accounts. It will show broker-wise holdings, stock names, quantities, invested amount, current value, profit/loss and trade history, along with current market prices, relevant news and a simple one-year estimated value.

The technical deliverables are:

– Python network engine for data collection

– TCP socket based IPC connection

– C++ OOP core with pointers and dynamic memory management

– Data parser using string streams

– Singly Linked List and Binary Search Tree (Phase III)

Progress Overview

The Python and C++ parts planned for Phase II are about 60 percent complete. The data collection layer, the C++ class design,

The Linked List and Binary Search Tree were deliberately kept for Phase III, so they are not counted as delays but tcp soket connection is a dalay

Codebase Information

Repository Link:
https://github.com/artsy-uday06/Cross-Broker-Real-Time-Portfolio-Aggregation-System

Testing and Validation Status

Test Type	Status (Pass/Fail)	Notes
no test conducted uptil now	NA	Will check that C++ receives all records sent by Python and data parsing yet to be conducted before phase III

Deliverables Progress

Completed: Python network engine (sample data); C++ OOP core with dynamic memory management.

In progress: Dashboard output; live broker data integration,tcp socket connection,data parsing

Pending: Singly Linked List and Binary Search Tree (Phase III); stock news and one-year projection; broker-wise comparison.